

WENGOLD
OPEN SILICON VERIFICATION

Architecture Specification

The open-source validity kernel for silicon verification

WENGOLD keeps the verification results that still hold after a change and reruns only what the change could have affected. It does this with three parts: one dependency graph of the design, a validity kernel that classifies every prior result against a change, and an append-only ledger of evidence. This document specifies those parts, the agent layer that acts on them, the tool surface they drive, and the boundary between the open core and the commercial distribution.

Field	Value
Document	Architecture Specification, v0.1
Status	Draft — pre-alpha reference for contributors
Licence	MIT or Apache-2.0 (open core)
Scope	Digital verification, RTL through gate level
Repository	github.com/wengold/wengold
Date	21 October 2026

Executive summary

Verification is the largest cost in a chip program — 40–50% of the budget on a leading-edge design, and heading toward 60–70% of engineering hours. Most of that spend is iteration, not first-pass work. A late change to the RTL, a register map or a clock-and-reset tree puts earlier results in doubt, and no one can say which passing tests, closed coverage, formal proofs and waivers are still valid for the new revision. Teams rerun whole regressions and re-close coverage they already closed.

WENGOLD answers that question in the open. It maintains one continuously updated dependency graph of the design, the verification environment and the runs, and a validity kernel that checks each earlier result against it. Results that still hold are kept, with the evidence behind them; results the change broke are thrown out; anything uncertain is re-established rather than assumed. Agents then run only the minimum new evidence the kernel asks for.

40–50% of a chip budget is verification today	**60–70%** of engineering hours is the trajectory	**3** verdicts per result: valid / invalidated / unknown	**1** graph the whole program shares

Why open source. The verification layer is where a small team is priced out of silicon. Every other part of the flow has an open option — Verilator, Yosys, SymbiYosys, cocotb, OpenROAD. The layer that decides what still needs re-checking does not. WENGOLD makes that layer open, so the five-person chip company can exist on a free toolchain.

The problem is iteration, not simulation

Faster simulators and single-task agents make each run cheaper. They do not reduce the number of runs, because the expensive question is not "how fast does this test go" but "do I still need to run it". After a change, the design and its verification environment are in an unknown relationship: some prior results are still valid, some are broken, and some cannot be judged from the change alone.

Without a way to tell those apart, the safe move is to rerun everything, and the cost of a change is the cost of the whole regression. That is the iteration tax, and it is what every faster tool still pays.

What the industry currently does

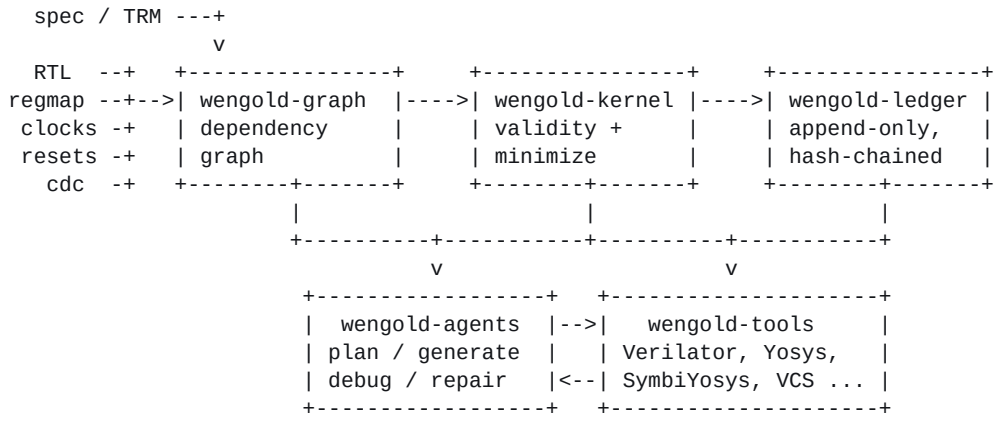
- **Rerun the regression.** Correct but expensive; the cost grows with the size of the design, not the size of the change.
- **Hand-pick tests.** Fast but unsound; the tests you skip are the ones that would have caught the bug.
- **Freeze the design late.** Trades schedule for safety; it is why verification drags to the end of a program.

Design principles

Principle	What it means in the system
Tools settle verdicts.	Agents propose; a deterministic check or a tool run decides. No verdict rests on a model's opinion alone.
Uncertainty is first-class.	A dependency the graph cannot resolve yields the verdict UNKNOWN, never a false pass. The kernel errs toward re-checking.
State is derived, never stored.	Whether a stage is done, stale or failed is computed from the ledger on demand, not kept as a flag that can drift.
Evidence is a witness.	A claim cannot exist without a real artifact — a log, waveform, counterexample or report — produced by a real tool and re-checkable later.
The engineer keeps signoff.	Agents act inside limits the team sets; high-consequence decisions and signoff stay with people.

System architecture

Three core services sit on one shared graph; an agent layer drives them; a tool layer settles every verdict. Data flows down the left into the graph, across into the kernel, and out through the ledger and the agents.



Component	Responsibility	Key output
wengold-graph	Build and maintain the dependency graph across design, verification environment and runs.	graph.tapeout + index
wengold-kernel	Classify every prior result against a change; compute the minimum new evidence.	verdicts + re-verify set
wengold-ledger	Record every run, revision, verdict and waiver, tamper-evidently.	ledger.tof
wengold-agents	Plan, generate tests, investigate failures, propose repairs.	test/repair proposals
wengold-tools	Drive simulators, formal engines and flow tools; settle verdicts.	logs, waveforms, reports

The dependency graph

One graph spans everything the program depends on, so a change anywhere can be traced everywhere. It is the substrate the kernel reads and the agents navigate.

Node kinds

RTL modules, register-map fields, tests, assertions, coverage points, runs, requirements, clock and reset roots, and CDC crossings.

Edge kinds

Edges are typed, because the kind of dependency decides how a change propagates: `drives`, `instantiates`, `verifies`, `covers`, `traces_to`, `clocked_by`, `reset_by`, `crosses`, and `unknown`.

> **The uncertain edge.** Where the builder cannot resolve a dependency it records an UNKNOWN edge. Anything downstream of one is classified UNKNOWN by the kernel, so an unresolved relationship becomes a re-check, never a silent pass.

Builders

Front-end	Source of truth	Backend
RTL parser	Module, port and instantiation structure	Surelog / sv-parser / Verible; regex fallback
Register map	Register and field access policy	SystemRDL / IP-XACT
Clock & reset	Clock and reset trees	RTL + SDC
CDC	Clock-domain crossings	Structural + formal

The validity kernel

The kernel is the heart of WENGOLD. On each change it answers one question for every prior result, and returns only what must be re-established. Everything still valid is carried forward with its evidence.

Verdict	Condition	Action
valid	No certain dependency on the change.	Keep the result and its evidence; do not rerun.
invalidated	Depends on, or is reached by, the change.	Rerun.
unknown	Reachable only across an uncertain dependency.	Re-establish; never assume.

The classification proceeds in four steps. A direct dependency on a changed node invalidates. Otherwise the kernel splits the result's upstream closure into what it is certain about and what is only reachable across an UNKNOWN edge: a certain reach invalidates, an uncertain reach yields UNKNOWN, and otherwise the result is valid.

Minimum new evidence

From the classification the kernel returns the smallest set of checks that gives a clear answer — the invalidated and unknown results — and keeps everything else. On a large SoC with thousands of changes a week, that difference decides how much verification runs again.

> **Worked example.** A one-line security bug in sec_firewall.sv invalidates the block test, the DMA-to-key test and the top-level smoke test that instantiates it; the NPU test is untouched and stays. The kernel reruns three results, not the regression.

Reference implementation

```
# wengold/kernel/validity.py
certain  = _reach_all(graph, result.depends_on, allow_unknown=False)
all_reach = _reach_all(graph, result.depends_on, allow_unknown=True)
uncertain_only = all_reach - certain
if result.depends_on & changed:  return INVALIDATED
if certain & changed:            return INVALIDATED
if uncertain_only & changed:     return UNKNOWN
return VALID
```

The ledger

The ledger is an append-only, hash-chained record of the program. Every run, revision, verdict and waiver is appended and chained to the entry before it, so the record is tamper-evident and any later reader can re-check it.

- **Append-only.** Entries are never edited; a correction is a new entry.
- **Hash-chained.** Each entry carries the hash of its predecessor; a tampered history fails verification.
- **Derived status.** Done / stale / failed is computed from the ledger on demand, never stored as a flag that can drift.
- **Evidence-bound.** Each verdict references the artifact that produced it, by digest.

> **Why it matters.** A ledger that cannot be edited is what lets a team trust a verdict it did not personally watch. It is also what lets the agent working on a chip change — a specialist model, an open-weight model, or a person — without the program's history leaving with it.

The agent layer

Agents decide where to look and which tool to run. They never write to the design directly: they propose, and the kernel and the tools decide. Four roles cover the loop.

Role	Job	Bounded by
Planner	Chooses what to re-establish and which tool settles each open result.	Team limits and PPA targets
Test generator	Writes tests, assertions and coverage for open items.	The plan and the spec
Debugger	Traces a failure to the relevant transaction across revisions.	Evidence in the ledger
Repairer	Proposes the smallest RTL or register-map change.	Re-verified by tools before it is trusted

An MCP server exposes the graph, kernel and ledger as tools, so any MCP-capable coding agent can drive WENGOLD from its own terminal.

Tool coverage

WENGOLD drives the tools a team already uses, in three groups. Open tools are first-class and work with no licence; commercial tools plug in through the same adapter contract. An absent binary is reported as `tool_missing`, never as a pass.

Group	Tools
01 · Simulation, debug and testbench	VCS · Xcelium · Questa · Verilator · Verdi · SimVision · Verisium · GTKWave · cocotb · UVM
02 · Formal, emulation and FPGA	JasperGold · VC Formal · OneSpin · ZeBu · Palladium · Veloce · Vivado · Quartus · Yosys · vManager
03 · Around the flow	Git · GitHub · GitLab · Jira · Slack · Tcl · Python · Make

The open backends that ship with the core are Verilator and Icarus for simulation, cocotb and UVM for testbenches, GTKWave for waveforms, and Yosys, SymbiYosys and Z3 for synthesis and formal proof. Every adapter exposes `available()` and `run()`, so the registry stays uniform across open and commercial tools.

Open core and the commercial distribution

The open core is everything needed to verify a design on a free toolchain. The commercial distribution adds the things a company needs to run it at scale, in a regulated environment, with support.

Open core — MIT or Apache-2.0	Commercial — WENGOLD Enterprise
Dependency graph, validity kernel, ledger	Hosted graph at fleet scale
Agent roles and MCP server	Curated agent packs and fine-tunes
Open tool adapters + commercial adapter contract	Air-gapped and on-prem deployment
CLI, Python API, CI integration	SSO, RBAC, audit export, support SLA
Community benchmarks and evals	Regression-farm orchestration

No capability that decides whether a design is correct is held back. The commercial layer sells scale, governance and support — not a better kernel.

Roadmap and non-goals

Release	Content
v0.1	Graph, kernel, ledger, tool registry, CLI, bundled SoC demo. (This scaffold.)
v0.2	LLM agents (plan / testgen / debug / repair), MCP server, cocotb runner.
v0.3	Register-map and CDC front-ends (SystemRDL, IP-XACT); Surelog and Verible backends.
v0.4	Formal loop (SymbiYosys + Z3) with counterexample-guided repair.
v0.5	Regression farm, coverage closure, waiver register.
v1.0	Spec-to-signoff on the open toolchain, reproducible on Sky130.

Non-goals

- WENGOLD does not claim foundry signoff. Missing tools and unpinned PDKs are reported as missing, never as passes.
- WENGOLD does not replace the engineer's judgement on high-consequence decisions; it carries the verification work around them.
- WENGOLD is not analog or mixed-signal today; that is on the roadmap, not in the core.

Get involved. The tool registry is the easiest first contribution: add a ToolSpec and a thin adapter. See CONTRIBUTING.md.